

3. Arrays

Una dimensión

No es posible almacenar más de un valor en una sola variable, a no ser que utilicemos un `array`.

Un `array` es un mapa ordenado donde los datos tendrán una clave (`key`) pero muchos valores (`values`). Por ejemplo, podríamos guardar los días de la semana bajo el mismo nombre de variable.

```
$semana = [  
    'Lunes',  
    'Martes',  
    'Miércoles',  
    'Jueves',  
    'Viernes',  
    'Sábado',  
    'Domingo'  
];
```

Los valores están dentro de corchetes (`[]`) separados por comas. De esta forma se asigna automáticamente un marcador que empieza por cero. Si quiero leer por separado cada elemento debo asignar el marcador envuelto en corchetes.

```
echo $semana[0];  
echo $semana[3];  
echo $semana[6];
```

Si quiero ver todo el contenido puedo usar la función nativa de PHP `var_dump`. Nos dirá la longitud del `array`, la posición de cada elemento, su tipo, longitud de cada valor y el propio valor que almacena.

```
var_dump($semana);
```

ES EN

`var_dump` no necesita añadir un `echo` delante.

Usa `var_dump` en lugar de `print_r`, proporciona más información.

Dentro de un array puede existir cualquier tipo, al igual que una variable.

```
$corteIngles = [  
    'Azul',  
    39,  
    True,  
    11  
];
```

Crear

Para declarar un `array` vacío solamente debemos crear una variable donde asignemos unos corchetes.

```
$planetas = [];
```

También lo puedes crear con su función: `$planetas = array();` .

Añadir

Y para ir añadiendo elementos.

```
$planetas[] = 'Marte';  
$planetas[] = 'Tierra';  
$planetas[] = 'Venus';
```

```
var_dump($planetas);
```

ES EN

Y si queremos utilizar un método más orientado a la programación funcional, podemos usar `array_merge` para crear un nuevo `array`.

```
$planetas = ['Marte', 'Tierra', 'Venus'];  
  
$nuevosPlanetas = array_merge($planetas, ['Mercurio']);  
  
var_dump($nuevosPlanetas);
```

Spread Operator (PHP 7.4+)

Desde PHP 7.4, existe una alternativa más moderna y eficiente a `array_merge`: el **spread operator** (`...`). Es más rápido y legible:

```
$planetas = ['Marte', 'Tierra', 'Venus'];  
  
$nuevosPlanetas = [...$planetas, 'Mercurio'];  
  
var_dump($nuevosPlanetas);
```

Puedes combinar múltiples arrays:

```
$terrestres = ['Tierra', 'Marte'];  
$gaseosos = ['Júpiter', 'Saturno'];  
$todos = [...$terrestres, ...$gaseosos];
```

También puedes añadir elementos en medio:

```
$inicio = ['primero', 'segundo'];  
$fin = ['cuarto', 'quinto'];  
$completo = [...$inicio, 'tercero', ...$fin];
```

ES EN

Una utilidad muy práctica para saber la longitud de un `array` es usar la funcion `count()` .

```
echo count($planetas);
```

Modificar

Para cambiar un valor hay que indicar la posición y el nuevo valor a introducir.
¿Recuerdas que el símbolo `=` es en realidad `←` ?

```
$planetas[2] = 'Saturno';
```

Borrar

Eliminar un elemento es un poco más marciano, debes usar una función nativa llamada `unset` . Personalmente creo que ha sido creada por un ser malvado. Me explico.

Supongamos que quiero destruir la Tierra antes de que lo haga el hombre. Es el 2º elemento, cuya posición es la 1.

```
unset($planetas[1]);
```

Veamos que ha pasado.

```
var_dump($planetas);
```

Se ha ido... pero ¡me ha desordenado las posiciones! Intentamos hacer un `Ctrl+Z` añadiendo la Tierra de nuevo.

```
$planetas[] = 'Tierra';  
  
var_dump($planetas);
```

Como dijo Jack Swigert en el Apolo 13: "-Houston, tenemos un problema-". Nuestro **array** pierde la posición que borramos para siempre. En realidad no es un **ES EN** problema porque la gran mayoría de las ocasiones lo recorreremos con un bucle (**loop**) e ignoraremos las posiciones, pero debes ser consciente de como funciona para no encontrarte sorpresas.

No obstante siempre puedes ignorar mis palabras y ordenarlo creando un nuevo array con un **loop** malgastando recursos.

```
unset($planetas[1]);

$planetasSinTierra = [];

foreach ($planetas as $posicion => $nombre) {
    $planetasSinTierra[] = $nombre;
}
```

Jugando con Strings

Observa y dime que ocurre.

```
$palabra = 'abcdef';
echo $palabra[2];
```

¿Qué ha pasado? Pues que los **string** pueden ser manipulados igual que un **array**.

```
$palabra = 'abcdef';
$palabra[2] = 'Z';
echo $palabra;
```

Un **string** se comporta como un **array** porque en el fondo no existen las palabras en la programación, sino **conjuntos de caracteres**. Dicho de otra manera: un **string** es un **array** de muchas letras.

Convertir un String en un Array

Andros Fenollosa 

ES EN

En algún momento tendrás la necesidad de pasar un texto a un **array** por medio de algún separador. Por ejemplo, transformar **una frase a array dividido por espacios**. Te dejo un ejemplo para realizarlo. El secreto esta en usar **preg_split**.

```
$frase = 'En un lugar de la Mancha';  
$arrayDeFrase = preg_split('/[\s,]+/', $frase);  
echo $arrayDeFrase[2];  
  
var_dump($arrayDeFrase);
```

No uses **split()**, la función esta OBSOLETA desde la versión 5.3.0. y ha sido ELIMINADA en la versión 7.0.0.

Diccionario

Las claves (**key**) pueden ser definidas por nosotros. A esto se le denomina **diccionario**.

```
$empleados = [  
    'Juan' => 34,  
    'Luisa' => 56  
];
```

Para **leer** será de la misma forma, salvo que ya no tenemos posiciones sino nuestras propias claves.

```
echo $empleados['Luisa'];
```

A la hora de **añadir** tendrás que indicar directamente el nombre que quieras darle.

```
$empleados = [];  
$empleados['Manolo'] = 99;  
var_dump($empleados);
```

Modificar será igual, indicando la clave.
Andros Fenollosa 

ES EN

```
$empleados['Manolo'] = 11;  
var_dump($empleados);
```

Y borrar de la misma forma que un `array`.

```
$empleados = [];  
$empleados['Manolo'] = 99;  
$empleados['Juan'] = 99;  
var_dump($empleados);  
  
unset($empleados['Manolo']);  
var_dump($empleados);
```

Dos dimensiones

Al principio de la lección dije que se podía añadir dentro de un `array` cualquier elemento, incluso... podríamos rizar el rizo insertando... ¡Otro `array`! A esto se le llama **array de dos dimensiones**, cuando un `array` tiene dentro otro `array`.

```
$rizo = [  
    []  
]
```

```
var_dump($rizo);
```

Tiene infinidad de usos. Pensemos que has heredado de un tío rico una tienda de ropa. Cada producto tiene su propio código de barras, precio, nombre, color y género.

```
$zara = [  
    123 => [  
        'nombre' => 'Camisa a cuadros',  
        'precio' => 29.95,  
        'sexo' => 'Hombre'  
    ],  
    234 => [  
        'nombre' => 'Falda manga',
```

Andros Fenolló a 

```
'precio' => 19.95,  
'sexo' => 'Mujer'  
],  
345 => [  
  'nombre' => 'Bolso minúsculo',  
  'precio' => 50,  
  'sexo' => 'Mujer'  
]  
];
```

ES EN

```
var_dump($zara);
```

El mecanismo para gestionarlo es igual al diccionario salvo que debemos ir nodo por nodo.

```
echo $zara[345]['nombre'];
```

Actividad 1
Actividad 2
Actividad 3
Actividad 4



Este trabajo está bajo una [licencia Attribution-NonCommercial-NoDerivatives 4.0 International](https://creativecommons.org/licenses/by-nc-nd/4.0/).



Desafíos de programación atemporales y multiparadigmáticos

Te encuentras ante un librito de actividades, divididas en 2 niveles de dificultad. Te enfrentarás a los casos más comunes que te puedes encontrar en pruebas técnicas o aprender conceptos elementales de programación.

Comprar el libro